

Vibe Coding – Lab 4 Chat History

Project: Space Invaders – Pygame

Student: Shreya Bonal

Purpose: Compiled record of the prompts and implementation steps used for the four Vibe Coding tasks in Lab 4.

1. Task 1 – Refine Collision Detection

Prompt used:

In this Pygame Space Invaders project, refine the player bullet collision detection. The current code removes a bullet from `self.player_bullets` while iterating directly over the same list, which can cause elements to be skipped and collisions to be missed. Modify the collision detection so that bullets are handled safely without modifying the list being iterated over. Ensure that each bullet can destroy an enemy only once, the bullet is removed after a successful collision, and the score increases exactly once for each destroyed enemy. Do not change unrelated game functionality.

Implementation: collision handling was changed to use a separate remaining-bullets list, preventing skipped elements and duplicate scoring.

2. Task 2 – Implement Game Over Condition

Prompt used:

Implement a proper Game Over condition and screen in this Pygame Space Invaders project. When the player is hit by an enemy bullet or the enemy grid reaches the bottom, stop normal gameplay and display a clear Game Over screen showing the final score. The game should wait for user input instead of only printing the result to the console. Keep the existing movement, shooting, collision detection, and scoring functionality unchanged. Do not implement the replay/difficulty feature yet because that is a separate task.

Implementation: a visible Game Over screen was added with the final score, and keyboard input allows the player to exit.

3. Task 3 – Add Replay Option and Difficulty

Prompt used:

Implement Task 3 from the README: Add a replay option to the Pygame Space Invaders game. After the Game Over screen, allow the player to choose whether to play again or exit. If the player chooses to play again, display a difficulty selection menu with Easy, Medium, and Hard options. Easy should have slower enemy movement and a lower enemy firing rate, Medium should use the default enemy speed and firing rate, and Hard should have faster enemy movement and a higher enemy firing rate. Starting a replay must create a fresh game with the selected difficulty and reset the score, player state, enemies, bullets, enemy movement direction, and other game state. Preserve the existing movement, shooting, collision detection, scoring, and Game Over functionality. Implement the replay and difficulty selection cleanly without changing unrelated functionality.

Implementation: Easy, Medium, and Hard difficulty settings were added, along with replay and a fresh game state for every replay.

4. Task 4 – Add Sound Feedback

Prompt used:

Implement Task 4 from the README: Add sound feedback to the Pygame Space Invaders game. Add a firing sound whenever the player successfully fires a bullet, an enemy-destroyed sound whenever a player bullet destroys an enemy, and a game-over sound when the game reaches the Game Over state. Initialize and use Pygame's sound functionality appropriately, and make sure the sounds are triggered only at the correct events and not repeatedly every frame. Keep the existing gameplay, movement, shooting, collision detection, scoring, Game Over, replay, and difficulty functionality unchanged. Add any required sound files and organize them appropriately within the project.

Implementation: sound feedback was added for firing, enemy destruction, and Game Over, with the Game Over sound protected from repeating every frame.

5. Final Repository and Videos

The completed code was committed and pushed to the student's SE Lab repository. The Lab 4 Videos folder contains the before and after recordings.

```
Lab 4/  
  46_space-invaders/  
  Videos/  
    Lab4_Before.mp4  
    Lab4_After.mp4
```

Final status: all four README tasks were implemented, the updated code was pushed, and both demonstration videos were uploaded to the repository.